THE SUPERIOR UNIVERSITY LAHORE

Assignment - 1 - Solution

Semester: 3rd **Section: C** **Session: Fall 2025**

Faculty of Computer Science and Information Technology **QCH:**

Subject: Data Structures and Algorithms **Total Marks: 20**

Name..……

Roll No..……

Instructions:

Write the solution of the given question in hard form and submit the Scan PDF to LMS before the deadline.

Task #	CLO #	Domain and BT Level	Total Points
1	1	C6	10
2	2	C3	10

Task 1: Integrated Document Management System with Undo and Sorting **(10 Marks)**

```
from collections import deque
class DocumentManager:
    def __init__(self):
        self.revisions = []

    def push_revision(self, revision):
        self.revisions.append(revision)
        print(f"Added: {revision}")

    def pop_revision(self):
        if self.revisions:
            removed = self.revisions.pop()
            print(f"Undo: Removed '{removed}'")
            if self.revisions:
                print(f"Current revision: {self.revisions[-1]}")
            else:
                print("No revisions left.")
        else:
            print("No revisions to undo.")

    def peek_revision(self):
        if self.revisions:
            print(f"Current revision: {self.revisions[-1]}")
        else:
            print("No revisions yet.")

    def show_history(self):
        print("Revision History:", self.revisions)
```

```
# ---------------- QUEUE: Print Jobs ----------------
class PrintScheduler:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, job):
        self.queue.append(job)
        print(f"Job with priority {job} added.")

    def dequeue(self):
        if self.queue:
            job = self.queue.popleft()
            print(f"Processing job with priority {job}")
        else:
            print("No jobs in queue.")

    def peek(self):
        if self.queue:
            print(f"Next job: {self.queue[0]}")
        else:
            print("Queue is empty.")

    def sort_jobs(self):
        jobs = list(self.queue)
        self.queue.clear()
        n = len(jobs)
        for i in range(n):
            swapped = False
            for j in range(0, n - i - 1):
                if jobs[j] > jobs[j + 1]:
                    jobs[j], jobs[j + 1] = jobs[j + 1], jobs[j]
                    swapped = True
            print(f"After pass {i+1}: {jobs}")
            if not swapped:
                break
        for job in jobs:
            self.queue.append(job)

# ----- Simulation -----
print("\n--- Document Management System ---")
doc = DocumentManager()
doc.push_revision("Version 1: Initial draft")
doc.push_revision("Version 2: Added chapter")
doc.push_revision("Version 3: Grammar fixes")
doc.show_history()
doc.pop_revision()
doc.show_history()
doc.pop_revision()
doc.show_history()
print("\n--- Print Scheduler ---")
scheduler = PrintScheduler()
for job in [5, 2, 8, 1]:
    scheduler.enqueue(job)
scheduler.sort_jobs()
while scheduler.queue:
    scheduler.dequeue()
```

Task 2: Event Planning System with Agenda Management and VIP Admissions **(10 Marks)**

```
from collections import deque

# ---------------- STACK: Session Agendas ----------------
class AgendaManager:
    def __init__(self):
        self.stack = []

    def push_session(self, session):
        self.stack.append(session)
        print(f"Added session: {session}")

    def pop_session(self):
        if self.stack:
            removed = self.stack.pop()
            print(f"Cancelled session: {removed}")
            if self.stack:
                print(f"Next session: {self.stack[-1]}")
            else:
                print("Agenda is empty now.")
        else:
            print("No sessions to cancel.")

    def peek_session(self):
        if self.stack:
            print(f"Next session: {self.stack[-1]}")
        else:
            print("Agenda is empty.")

    def is_empty(self):
        return len(self.stack) == 0

    def show_agenda(self):
        print("Agenda:", self.stack)


# ---------------- QUEUE: Attendee Admissions ----------------
class AdmissionQueue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, attendee):
        self.queue.append(attendee)
        print(f"Attendee {attendee} added.")

    def dequeue(self):
        if self.queue:
            person = self.queue.popleft()
            print(f"Admitted: {person}")
        else:
            print("No attendees waiting.")
```

```
    def peek(self):
        if self.queue:
            print(f"Next attendee: {self.queue[0]}")
        else:
            print("No attendees waiting.")

    def size(self):
        return len(self.queue)

    def sort_attendees(self):
        attendees = list(self.queue)
        self.queue.clear()
        n = len(attendees)
        for i in range(n):
            swapped = False
            for j in range(0, n - i - 1):
                if attendees[j][1] < attendees[j + 1][1]:  # Descending VIP
                    attendees[j], attendees[j + 1] = attendees[j + 1], attendees[j]
                    swapped = True
            print(f"After pass {i+1}: {attendees}")
            if not swapped:
                break
        for person in attendees:
            self.queue.append(person)


# ----- Simulation -----
print("\n--- Event Planning System ---")

agenda = AgendaManager()
agenda.push_session("Session 1: Keynote")
agenda.push_session("Session 2: AI Panel")
agenda.push_session("Session 3: Networking")
agenda.show_agenda()

agenda.pop_session()
agenda.show_agenda()
agenda.pop_session()
agenda.show_agenda()
print("Is agenda empty?", agenda.is_empty())

print("\n--- VIP Admissions ---")
admissions = AdmissionQueue()
admissions.enqueue(("Alice", 3))
admissions.enqueue(("Bob", 1))
admissions.enqueue(("Charlie", 5))
admissions.enqueue(("Diana", 2))

admissions.sort_attendees()

while admissions.queue:
    admissions.dequeue()
    print("Remaining size:", admissions.size())
```